# Validation, Pagination, Filtering & Sorting



## Why it matters

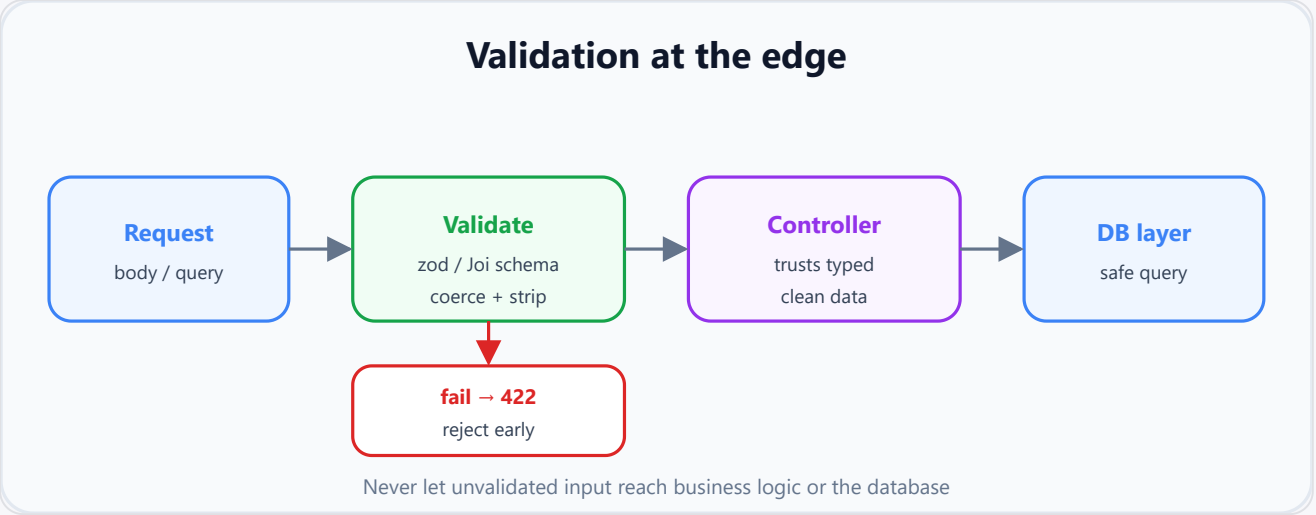
Every input to your API is a potential attack, a potential crash, and a potential data-corruption bug. "Never trust the client" is the first law of backend engineering, and **validation** is how you enforce it. An endpoint that reads `req.body.age` and stores it without checking will happily accept `"age": "banana"`, `"age": -5`, or a 10-megabyte string designed to exhaust your memory. Validation at the edge is what turns a fragile endpoint into a robust one, and it is the single highest-leverage habit separating hobby code from production code.

**Pagination** is equally load-bearing but for a different reason: scale. An endpoint that returns *all* users works fine with 100 rows and takes your database down with 10 million. `GET /users` without pagination is one of the most common causes of production incidents, because it passes every test on a small dev database and detonates in production. Interviewers ask "how would you paginate this?" precisely because the naive answer (offset/limit) has a subtle scaling flaw that only shows up at volume, and knowing the cursor-based alternative signals real experience.

Filtering and sorting round out the read story: real clients need `?status=active&sort=-createdAt`, and doing this cleanly — without opening a SQL-injection hole or letting clients sort by an unindexed column — is a craft worth knowing.

## The concept, simply

**Validation** means checking, at the boundary of your system, that incoming data has the right shape, types, and constraints *before* any business logic touches it. You declare a *schema* — "email must be a valid email, age must be an integer 0–150, role must be one




Validate at the edge. Clean, typed data reaches your controller; invalid input is rejected early with 422.

Three libraries dominate the Node ecosystem: **Joi** (mature, chainable, framework-agnostic), **Zod** (TypeScript-first, infers static types from the schema — currently the most popular in new code), and **express-validator** (middleware built around express and the validator.js rules). They differ in ergonomics but share the idea: define the schema once, validate against it, get typed and coerced data out.

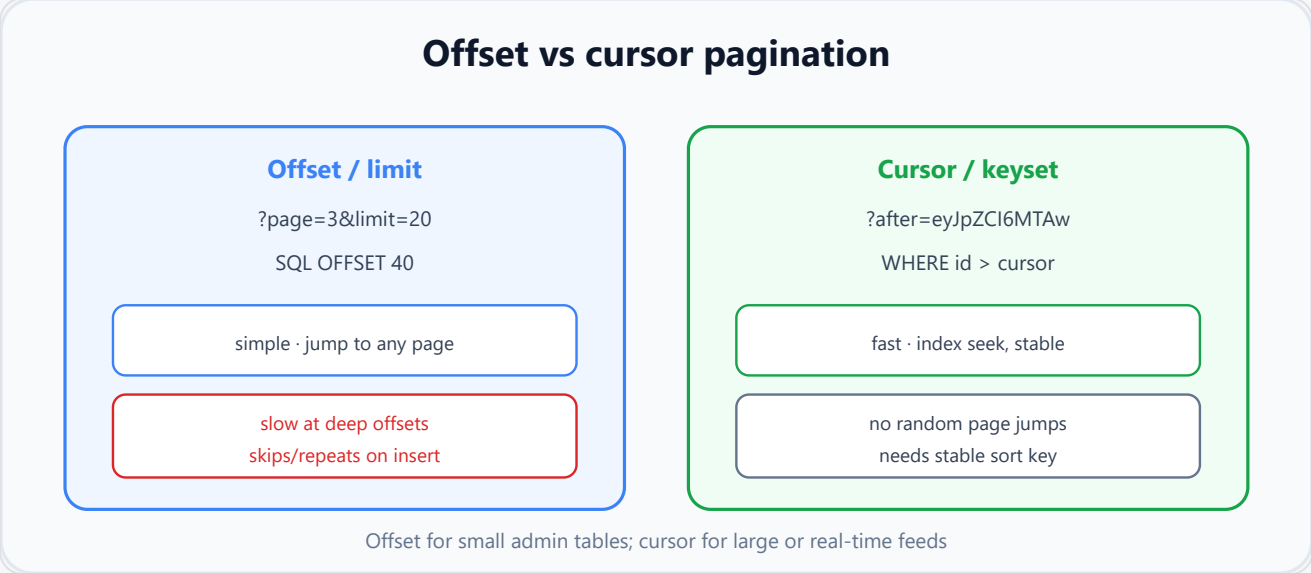
**Pagination** means returning results in bounded chunks instead of all at once. There are two families: *offset-based* (`?page=3&limit=20`) and *cursor-based* (`?after=<opaque-cursor>`). **Filtering** narrows results by field values (`?status=active`); **sorting** orders them (`?sort=-createdAt` for newest first).

## How it works (deep)

**Validation as a gate.** The correct architecture is a validation *middleware* that runs before the controller. It parses the request against the schema; on success it replaces `req.body` with the *coerced, stripped* value (unknown keys removed, `"25"` turned into `25`); on failure it short-circuits with a 422 and a list of what was wrong. The controller then trusts its input completely — no defensive `if (typeof x !== 'number')` scattered through business logic. This separation is the whole win: validation lives in one declarative place, not smeared across every handler.

A subtle but important point: **validate query and params too, not just the body.** `?limit=999999` or `?limit=abc` must be caught. A common bug is validating the JSON body carefully while blindly trusting `req.query.limit`, then passing it straight to the database as a `LIMIT`. Always coerce and clamp pagination params.

**Offset pagination** is the intuitive one: `LIMIT 20 OFFSET 40` skips 40 rows and returns the next 20. It lets clients jump to any page and is trivial to implement. It has two real flaws at scale.




Offset is simple but slows down and drifts at scale; cursor pagination is fast and stable but can't jump to arbitrary pages.

Flaw one: *performance*. `OFFSET 1000000` forces the database to walk and discard a million rows before returning your twenty — it gets linearly slower the deeper you page. Flaw two: *drift*. If a new row is inserted while a user pages through results, the offsets shift, so they can see a row twice or skip one entirely. For a stable, high-volume feed this is unacceptable.

**Cursor (keyset) pagination** fixes both. Instead of "skip N rows," you say "give me rows *after* this specific point," where the point is encoded in an opaque cursor (often a base64 of the last row's sort key + id). The query becomes `WHERE (created_at, id) < (:cursor_ts, :cursor_id) ORDER BY created_at DESC, id DESC LIMIT 20`. Because it uses an index seek rather than a scan-and-discard, it stays fast no matter how deep you go, and because it anchors to a real row rather than a numeric offset, inserts don't cause skips or repeats. The trade-off: you cannot jump to "page 47" — only forward/backward

**Filtering and sorting safely.** The danger is letting arbitrary client strings reach your query. Never interpolate `req.query.sort` into raw SQL — that is an injection hole. Instead, maintain an *allow-list*: map the client's `?sort=-createdAt` to a known column and direction, and reject anything not on the list. Same for filters — validate field names against a whitelist and use parameterized queries or your ORM's query builder for the values. Also restrict sorting to *indexed* columns; letting a client sort a ten-million-row table by an unindexed field is a self-inflicted denial of service.

## Code examples

A reusable Zod validation middleware — declare the schema once, get clean data or a 422.

JAVASCRIPT Copy

```
const { z } = require('zod');

// Generic middleware: validate a chosen part of the request
function validate(schema, source = 'body') {
  return (req, res, next) => {
    const result = schema.safeParse(req[source]);
    if (!result.success) {
      return res.status(422).json({
        error: 'Validation failed',
        details: result.error.issues.map(i => ({
          field: i.path.join('.'),
          message: i.message,
        })),
      });
    }
    req[source] = result.data; // coerced + stripped — controller trusts t
    next();
  };
}

const createUserSchema = z.object({
  email: z.string().email(),
  age: z.coerce.number().int().min(0).max(150),
```

```
router.post('/users', validate(createUserSchema), createUser);
```

Validating and clamping pagination query params so a client can never demand a million rows:

JAVASCRIPT Copy

```
const pageQuerySchema = z.object({
  page:  z.coerce.number().int().min(1).default(1),
  limit: z.coerce.number().int().min(1).max(100).default(20), // hard ceil
  sort:  z.string().optional(),
  status: z.enum(['active', 'inactive']).optional(),
});

router.get('/users', validate(pageQuerySchema, 'query'), listUsers);
```

Offset pagination with a metadata envelope so clients know how to navigate:

JAVASCRIPT Copy

```
async function listUsers(req, res) {
  const { page, limit, status } = req.query;
  const filter = status ? { status } : {};

  const [items, total] = await Promise.all([
    User.find(filter).skip((page - 1) * limit).limit(limit),
    User.countDocuments(filter),
  ]);

  res.json({
    data: items,
    pagination: {
      page, limit, total,
      totalPages: Math.ceil(total / limit),
      hasNext: page * limit < total,
    },
```

Cursor pagination with safe, allow-listed sorting — the version you reach for at scale:

JAVASCRIPT Copy

```javascript
// Allow-list maps client tokens to real, indexed columns + direction
const SORTABLE = { createdAt: 'created_at', name: 'name' };

async function feed(req, res) {
  const limit = Math.min(Number(req.query.limit) || 20, 100);
  const cursor = req.query.after
    ? JSON.parse(Buffer.from(req.query.after, 'base64').toString())
    : null;

  // Keyset: fetch rows strictly "after" the cursor's (createdAt, id)
  const rows = await Post.findAfter(cursor, limit + 1); // +1 to detect mo
  const hasMore = rows.length > limit;
  const page = rows.slice(0, limit);

  const last = page[page.length - 1];
  const nextCursor = hasMore
    ? Buffer.from(JSON.stringify({ createdAt: last.createdAt, id: last.id
    : null;

  res.json({ data: page, nextCursor });
}
```

## Common mistakes & gotchas

- **Only validating the body:** query and path params are input too. `?limit=99999999` and `/users/'; DROP TABLE` must be checked.
- **No upper bound on `limit`:** a client sending `?limit=1000000` can exhaust memory and stall the DB. Always clamp with a hard max.
- **Deep offset pagination:** `OFFSET 500000` is slow and drifts under concurrent inserts. Use cursors for large/real-time datasets.

- **Interpolating sort/filter fields into SQL:** a direct injection vulnerability. Allow-list field names; parameterize values.
- **Sorting on unindexed columns:** lets a client trigger full-table sorts — effectively a DoS.
- **Trusting client-coerced types:** `req.query.limit` is a *string*; `"20" - 1` works but `"20" + 1` is `"201"`. Coerce explicitly.
- **Leaking internal error detail:** return which fields failed, not stack traces or DB messages.
- **Validating in the controller by hand:** scatters rules and drifts. Centralize in schema middleware.

## Best practices

- Validate **every** input surface — body, query, params — with a declarative schema (Zod/Joi/express-validator) in middleware, before the controller.
- **Coerce and strip**: turn strings into the right types and remove unknown keys so the controller trusts its input.
- Return **422** with a structured list of field-level errors, never a stack trace.
- **Clamp pagination**: default and maximum `limit`, minimum `page`/positive cursor.
- Prefer **cursor pagination** for large or frequently-changing datasets; offset is fine for small, stable admin tables.
- **Allow-list** sortable and filterable fields, map them to indexed columns, and parameterize all values.
- Return a **consistent envelope** (`{ data, pagination }` or `{ data, nextCursor }`) so clients paginate uniformly.

## ⭐ Interview lens

> **Q: Where should validation happen, and why there?**
> At the edge, in middleware before the controller, so invalid input never reaches business logic or the database. Centralizing it in a schema keeps rules in one

declarative place and lets the controller trust coerced, stripped data. Validating inside handlers scatters and drifts.

**Q: Offset vs cursor pagination — when and why?**

Offset ( `page/limit` ) is simple and allows jumping to any page but slows down at deep offsets (scan-and-discard) and drifts when rows are inserted mid-scroll. Cursor/keyset anchors to the last row's sort key, uses an index seek so it stays fast at any depth, and doesn't skip/repeat under inserts — but can't jump to arbitrary pages. Use cursors for large or real-time feeds.

**Q: A client sends `?sort=passwordHash`. What's the risk and the fix?**

If you interpolate it into a query you get injection and you may expose or sort by sensitive/unindexed columns. Fix: an allow-list mapping client tokens to known indexed columns and directions, rejecting anything else, with parameterized queries for values.

**Q: Which status code for a validation failure — 400 or 422?**

422 Unprocessable Entity is more precise: the request parsed fine but failed semantic rules. 400 is for syntactically broken requests (unparseable JSON). Either is defensible if consistent, but 422 shows you know the distinction.

**Q: Why cap `limit`, and what happens if you don't?**

Without a ceiling, a client can request millions of rows, exhausting server memory and stalling the DB — a trivial DoS. Always clamp with a sane default (e.g. 20) and hard max (e.g. 100).

**Q: Zod vs Joi vs express-validator?**

All validate at the edge. Zod is TypeScript-first and infers static types from the schema (great for TS codebases). Joi is mature and framework-agnostic with a

failure — is identical.

## Cheat-card

- **Never trust the client.** Validate body + query + params with a schema in middleware, before the controller.
- **Coerce, strip, clamp:** types fixed, unknown keys removed, `limit` bounded.
- Validation fail → **422** with field-level details (no stack traces).
- **Offset** = `page/limit`: simple, jumps pages, but slow + drifts at scale.
- **Cursor/keyset** = "rows after X": fast at any depth, stable under inserts, no random jumps.
- **Allow-list** sort/filter fields → indexed columns; parameterize values (no SQL injection).
- Return a consistent envelope: `{ data, pagination }` or `{ data, nextCursor }`.
- Libraries: **Zod** (TS-first), **Joi** (mature), **express-validator** (middleware).

QUICK CHECK — no marks, just for you.

1. Where should request validation run, and why there?

- ◯ Inside each controller by hand, so rules stay close to where data is used
- ◯ Only in the database via column constraints, so app code stays simple
- ◯ At the edge in middleware before the controller, so invalid input never reaches business logic or the database
- ◯ On the client only, since the server can trust data the UI already checked

Check answer

- It cannot sort results and it always returns duplicate rows
- It requires a cursor and it breaks HTTP caching
- It exposes SQL injection and it ignores the LIMIT clause
- It gets linearly slower because the DB walks and discards skipped rows, and offsets drift when rows are inserted mid-scroll

Check answer

3. A client sends ?sort=passwordHash. What is the safe way to handle sort/filter fields?

- Maintain an allow-list mapping client tokens to known indexed columns and directions, rejecting anything else
- Interpolate the field name directly into the SQL ORDER BY clause
- Trust the field but wrap the whole query in a try/catch
- Allow any column but cap the result limit to 100 rows

Check answer

‹ Previous
CRUD, HTTP Verbs, Status Codes & Idempotency

Next ›
API Versioning, Error Shape, Rate Limiting & Docs

LEARN MORE

ARTICLES & DOCS

express-validator - Getting Started
express-validator.github.io

cursor.skip() - MongoDB Manual (Pagination)
www.mongodb.com

Mongoose Queries (limit, skip, sort)
mongoosejs.com

VIDEOS




Create A Paginated API With Node.js - Complete Tutorial
Web Dev Simplified

Hand-picked references for this lesson.